Laporan Tugas Kecil 2

IF2211 Strategi Algoritma

Voxelization Objek 3D Menggunakan Octree

Semester II Tahun 2025/2026

Edward David Rumahorbo	13524036
Narendra Dharma Wistara M.	13524044

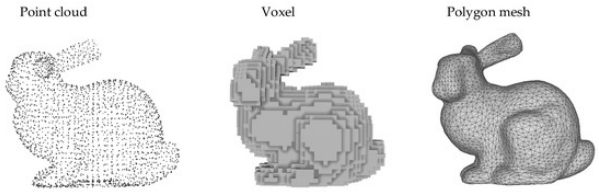
Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

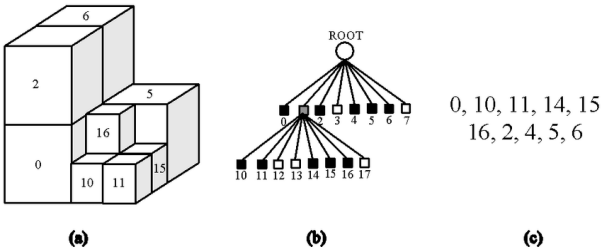
Daftar Isi

1 Deskripsi Penugasan



Gambar 1: Visualisasi *Voxelization*
Sumber: https://www.mdpi.com/1424-8220/22/18/7040

Voxelization merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil seperti di Minecraft. Proses konversi ini dilakukan dengan memanfaatkan struktur data **Octree**, yaitu sebuah *tree* yang setiap percabangan terdiri dari 8 anak. Melalui *Octree*, sebuah ruang kubus 3D dapat dipotong menjadi 8 kubus berbeda, yang masing-masing dapat dipotong lagi menjadi 8 kubus yang lebih kecil.

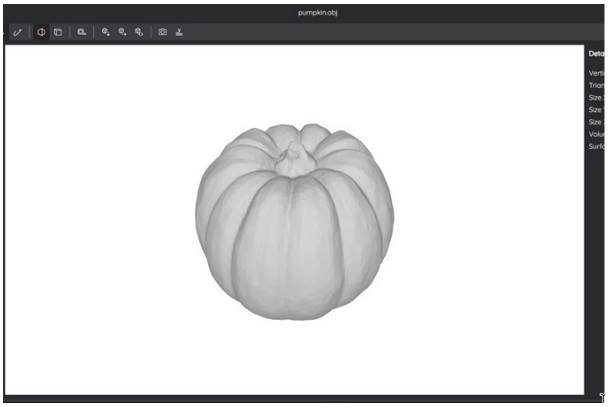


Gambar 2: Visualisasi *Octree* terhadap 3D Model
Sumber:
https://www.researchgate.net/figure/a-Octants-b-Octree-Structure-c-Linear-Octree-Each-of-the-cubes-has-a-locational-code_fig1_221115507

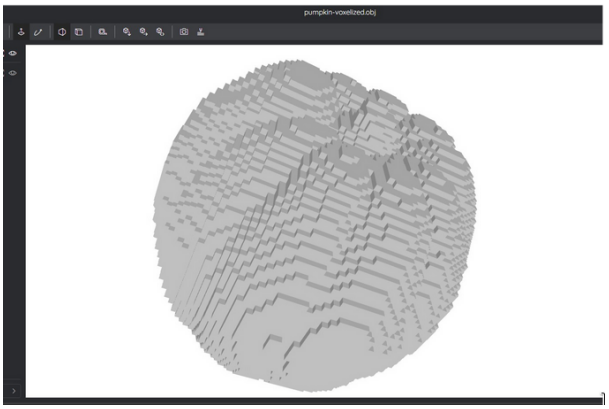
Tugas Anda adalah mengimplementasikan sebuah program yang dapat mengkonversi model .obj yang umum menjadi model .obj yang terdiri dari *voxel* dengan memanfaatkan properti *Octree*. Program melakukan konversi model dengan menggunakan algoritma ***divide and conquer***.

2 Ilustrasi Kasus

Diberikan sebuah model 3D sebagai berikut.



Di bawah ini adalah hasil perubahan.



3 Spesifikasi Tugas

3.1 Spesifikasi Wajib

- Buatlah sebuah program dalam bahasa **Go, C, C++, atau Java** yang mengimplementasikan **algoritma *Divide and Conquer*** untuk melakukan konversi model tiga dimensi.
- Output voxel harus berukuran **uniform**, diambil dari *leaf* pada *depth* maksimum.
- Transformasi hanya perlu dilakukan pada permukaan objek.
- Tugas dikerjakan **berkelompok** dengan anggota **2 orang**, boleh lintas kelas maupun kampus.
- **Input:** Program menerima *path* dari sebuah file `.obj` yang ingin dikonversi dan juga tingkat kedalaman pohon maksimum. Untuk tugas besar ini, **hanya memperhatikan nilai x, y, z pada *vertice* v, dan *faces* f.** Program harus bisa menentukan apakah input valid atau tidak. Format `.obj` valid adalah *row-row* yang berisikan `<v x y z>` atau `<f i j k>`.

  Contoh konten `.obj`:

  ```
  v  2.229345 -0.992723 -0.862826
  v  2.292449 -0.871852 -0.882400
  v  2.410367 -0.777999 -0.841105
  f 1 2 3
  ```

- **Output:** Program mengembalikan:
  - sebuah file `.obj` yang merupakan hasil konversi;
  - Di CLI, program harus menampilkan setidaknya informasi-informasi berikut:
    * Banyaknya voxel yang terbentuk
    * Banyaknya vertex yang terbentuk
    * Banyaknya faces yang terbentuk
    * Statistik *node octree* yang terbentuk dalam format:

      ```
      1 : <banyaknya node dengan depth 1 yang terbentuk>
      2 : <banyaknya node dengan depth 2 yang terbentuk>
      ...
      N : <banyaknya node dengan depth n yang terbentuk>
      ```

    * Statistik node yang **tidak perlu** ditelusuri dalam format:

```
1 : <banyaknya node dengan depth 1 yang tidak perlu ditelusuri>
2 : <banyaknya node dengan depth 2 yang tidak perlu ditelusuri>
...
N : <banyaknya node dengan depth n yang tidak perlu ditelusuri>
```

* Kedalaman *octree*
* Lama waktu program berjalan
* *Path* dimana file `.obj` disimpan

3.2 Spesifikasi Bonus

Poin maksimal untuk bonus adalah 10. Silahkan pilih spesifikasi bonus yang akan dikerjakan.

- Menggunakan *concurrency* dalam bahasa pemrograman yang dipilih untuk algoritma konversi (max 5 poin)
- Mengimplementasikan `.obj` viewer minimal menampilkan model 3D hasil *voxelization* secara interaktif dalam bahasa pemrograman yang dipilih untuk menampilkan hasil objek yang dikonversi (max 10 poin)

4 Dasar Teori

4.1 Algoritma Divide and Conquer

Algoritma Divide and Conquer merupakan strategi penyelesaian masalah yang bekerja dengan membagi suatu persoalan besar menjadi beberapa upa-persoalan yang lebih kecil namun memiliki karakteristik yang serupa dengan persoalan semula. Proses ini dilakukan secara rekursif melalui tiga tahapan utama, yaitu Divide untuk membagi masukan, Conquer untuk menyelesaikan upa-persoalan secara langsung atau rekursif, serta Combine untuk menggabungkan kembali solusi-solusi tersebut menjadi satu kesatuan solusi utuh.

4.2 Voxelization

Voxelization merupakan metode transformasi objek tiga dimensi yang bersifat kontinu, menjadi model tiga dimensi yang tersusun atas kubus-kubus kecil yang disebut dengan *voxel*. Proses ini bertujuan agar objek dapat diolah lebih mudah direpresentasikan di ruang digital.

4.3 Octree

Struktur data Octree merupakan representasi pohon di mana setiap simpul internal memiliki tepat delapan anak. Masing-masing simpul ini melambangkan bagian dari sebuah bangun ruang tiga dimensi. Sebuah kubik dapat dipotong menjadi delapan subkubus yang lebih kecil lagi sehingga tiap bagian tersebut mencapai tingkat kedalaman yang diinginkan.

4.4 File .obj

Format file .obj adalah format representasi yang menyimpan data model tiga dimensi. Informasi utama yang disimpan dalam format .obj ini merupakan koordinat verteks (v) yang menentukan titik-titik dalam model dan definisi faces (f) yang menyusun permukaan poligon dari verteks yang tersedia.

Berikut merupakan contoh representasi konten dalam file .obj.

```
v 2.229345 -0.992723 -0.862826
v 2.292449 -0.871852 -0.882400
v 2.410367 -0.777999 -0.841105
f 1 2 3
```

4.5 Concurrency dalam Go

Penerapan *concurrency* dalam bahasa pemrograman Go memungkinkan pengolahan data *Octree* dilakukan secara paralel, sehingga mengoptimalkan penggunaan sumber daya perangkat keras pada sistem multi-inti. Berbeda dengan model *multithreading* pada Java yang menggunakan objek *Thread* dengan alokasi memori besar, Go menggunakan *goroutines* yang sangat ringan dan dikelola oleh *runtime* Go[cite: 3]. Sinkronisasi antar proses dilakukan menggunakan `sync.WaitGroup` untuk koordinasi penyelesaian tugas dan `sync.Mutex` untuk melindungi integritas data statistik bersama dari fenomena *race condition*[cite: 3].

Concurrency merupakan kemampuan program untuk menangani berbagai tugas secara bersamaan melalui dekomposisi proses yang independen. Dalam ekosistem Go, konsep ini diimplementasikan melalui goroutines. Hal ini memungkinkan sebuah aplikasi menjalankan ribuan hingga jutaan eksekusi paralel tanpa membebani sistem secara signifikan.

Berikut adalah potongan implementasi untuk *concurrency* dalam bahasa Go.

```go
go func() {
    // Sinkronisasi menggunakan WaitGroup
    var wg sync.WaitGroup
    wg.Add(1)

    // Pemanggilan fungsi secara concurrent
    go BuildOctreeConcurrent(node, vertices, faces, currentDepth, maxDepth,
        stats, &wg)

    // Menunggu seluruh proses selesai
    wg.Wait()
}
```

5 Desain

5.1 Tech Stack

Aplikasi ini dirancang menggunakan bahasa Go untuk memudahkan pengimplementasian *Concurrency* (Spesifikasi Bonus), serta memanfaatkan sintaks bahasa Go yang mudah dipahami dan mudah menemukan error. Secara garis besar, arsitektur aplikasi terdiri dari komponen-komponen berikut.

1. Bahasa Pemrograman: Go 1.25.5
2. Lingkungan Pengembangan: Linux

5.2 Struktur Program

Proyek disusun mengikuti standar Maven dan spesifikasi tugas:

1. `src/`: Kode sumber program.
2. `test/`: Masukan yang digunakan untuk *testing*
3. `bin/`: *File executable* atau hasil kompilasi.
4. `doc/`: Spesifikasi dan laporan tugas kecil dalam PDF

Program utama dibagi menjadi dua komponen utama untuk memisahkan algoritma dengan GUI:

- `main.go` berisi *entry-point* program yang memanggil fungsi-fungsi lain yang digunakan.
- `octree.go` berisi struktur data *Octree* dan pembangunan *bounding box*
- `build.go` berisi pengimplementasian algoritma *Divide and Conquer* untuk membangun struktur *Octree*, serta penggunaan *concurrency*.
- `parser.go` berisi logika *parsing* untuk file .obj.
- `viewer.go` berisi pengimplementasian fitur bonus untuk 3D *object viewer*.
- `optimizer.go` berisi logika *optimizing* agar *face* yang muncul tidak duplikat dan sisi kubus yang tertutup kubus lain tidak perlu dimasukkan.

5.3 Cara Kompilasi dan Menjalankan Program

Untuk mengompilasi seluruh kode sumber dan mengunduh dependensi Go, jalankan perintah berikut pada direktori *src/*.

```
go mod tidy
```

Untuk melakukan kompilasi (*build*) program menjadi *file executable*, gunakan perintah berikut.

```
go build -o voxelizer src/*.go
```

Untuk menjalankan *file executable*, gunakan perintah berikut.

```
./voxelizer
```

Atau jika ingin menjalankan program tanpa mengompilasi, gunakan perintah berikut.

```
go run src/*.go
```

6 Implementasi Spesifikasi Wajib

6.1 Algoritma Utama

Algoritma UtamaAlgoritma ini akan mencoba untuk menempatkan satu ratu pada setiap baris secara sistematis dan memeriksa validitasnya. Untuk membantu algoritma ini, dibuat beberapa struktur data, yaitu:

- `Board` digunakan sebagai representasi fisik dari papan permainan. Struktur data ini diimplementasikan menggunakan matriks atau *array* dua dimensi. Setiap elemen dalam *array* menyimpan warna dalam petak.

```
public static String[][] board;
```

- `Koordinat` digunakan untuk menyimpan koordinat pada *board*. `Koordinat` diimplementasikan menggunakan struktur data *record* dari Java. Setiap *record* menyimpan koordinat dalam papan dengan x sebagai baris dan y sebagai kolom.

```
public record Koordinat(int x, int y) {}
```

- `Koordinat[] queenLocation` digunakan untuk menyimpan *set of queens* yang sedang divalidasi.

```
Koordinat[] queenLocation = new Koordinat[n];
for (int i = 0; i < queenLocation.length; i++) {
    queenLocation[i] = new Koordinat(i, 0);
}
```

- *HashSet* `uniqueColor` yang digunakan untuk menyimpan setiap warna yang ada di papan untuk dihitung jumlah warna yang terdapat di dalam papan.

```
Set<String> uniqueColor = new HashSet<>();
```

Berikut adalah langkah-langkah algoritmanya:

1. Program membuat *array of Record* `queenLocation` berukuran N. Di mana setiap elemen array menyimpan koordinat (x, y), di mana x adalah baris $[0..N-1]$ dan y adalah kolom yang awalnya diinisialisasi dengan 0.
2. Program memasuki *infinite loop* dengan `while (true)` untuk mengevaluasi setiap kemungkinan penempatan ratu. Pada setiap iterasi, program mengecek apakah konfigurasi ratu saat ini sudah memenuhi aturan-aturan berikut.
   - Tidak ada dua ratu di kolom yang sama
   - Tidak ada dua ratu di wilayah warna yang sama
   - Tidak ada dua ratu yang saling bersebelahan, termasuk secara diagonal
3. Jika konfigurasi memenuhi aturan, maka algoritma berhenti dan mengembalikan lokasi ratu.

4. Jika konfigurasi tidak memenuhi aturan, maka program akan menggeser ratu
    - Mulai dari baris pertama. Jika kolom belum mencapai indeks maksimal $N - 1$, geser kolom ratu ke bawah sebesar 1 petak.
    - Jika kolom sudah mencapai indeks maksimal, kembalikan kolom ke 0, lalu pindah ke baris berikutnya.
    - Ulangi hingga seluruh kemungkinan teriterasi atau solusi ditemukan.
5. Jika tidak ditemukan solusi hingga akhir, maka program berhenti dan menyatakan tidak ada soluti.

Algoritma ini memiliki kompleksitas waktu algoritma sebesar $O(N^N$, sehingga untuk N yang sangat besar memakan waktu yang sangat lama.

6.2 Algoritma Utama dalam Java

Algoritma Utama dalam Java

```java
public static boolean solve(Koordinat[] queenLocation, Consumer<Koordinat[]>
    onStep) {
    int n = board.length;

    while (true) {
        totalKasus++;

        if (validSolution(queenLocation)) {
            return true;
        }

        int baris = 0;
        while (baris < n) {
            int kolom = queenLocation[baris].y;
            if (kolom < n - 1) {
                queenLocation[baris] = new Koordinat(baris, kolom + 1);
                break;
            } else {
                queenLocation[baris] = new Koordinat(baris, 0);
                baris++;
            }
        }

        if (baris == n) {
            break;
        }
    }
    return false;
}
```

6.3 Fungsi Pembantu

Untuk membantu berjalannya program utama, digunakan beberapa fungsi pembantu seperti.

1. `validSolution()`. Fungsi ini mengambil input berupa *list of Record* `Koordinat[] queenLocation` dan mengembalikan `boolean`. Fungsi ini berfungsi untuk mengecek apakah konfigurasi *queens* saat ini sudah memenuhi syarat.

    ```java
    public static boolean validSolution(Koordinat[] queenLocation) { ... }
    ```

2. `checkColor()`. Fungsi ini mengambil sebuah *record* dan mengembalikan `string` yang berisi label warna dari sebuah koordinat.

    ```java
    public static String checkColor(Koordinat queen) { ... }
    ```

3. `colorCount()`. Fungsi ini mengembalikan `integer` jumlah total warna yang ada dalam papan.

```java
    public static int colorCount() {
        Set<String> uniqueColor = new HashSet<>();
        ...
        return uniqueColor.size();
    }
```

4. `loadBoard()`. Fungsi ini mengambil sebuah `string` berisi *path file* `.txt` dan mengembalikan `boolean` apakah *file* tersebut merupakan konfigurasi papan yang valid.

```java
    public static boolean loadBoard(String filePath) {
        ...
        return board.length == colorCount();
    }
```

5. `printBoard()`. Fungsi ini mengambil sebuah konfigurasi ratu dan mencetak papan ke terminal.

```java
    public static void printBoard(Koordinat[] queenLocation) {
       if (isQueen) System.out.print("#");
        else System.out.print(board[i][j]);
    }
```

7 Implementasi Spesifikasi Bonus

7.1 Concurrency pada Pembangunan Octree

Algoritma *Divide and Conquer* bekerja dengan cara membagi satu kotak (*node*) menjadi delapan kotak anak. Proses pengecekan pada setiap kotak anak ini berdiri sendiri dan tidak saling bergantung satu sama lain. Karena alasan tersebut, proses ini sangat cocok dipercepat menggunakan *concurrency* (pemrosesan bersamaan).

Pada program ini, *concurrency* dibuat menggunakan *Goroutines* yang merupakan fitur bawaan dari bahasa Go. Agar proses pemrosesan berjalan aman, digunakan dua alat bantu dari pustaka `sync`:

- `sync.WaitGroup`: Alat ini bertugas menahan atau menyuruh proses induk untuk menunggu sampai kedelapan anak *node*-nya selesai diproses. Setelah semua anak selesai, barulah program lanjut ke tahap berikutnya.
- `sync.Mutex`: Alat ini digunakan untuk mencegah tabrakan data (*race condition*). Karena ada banyak *goroutine* yang berjalan bersamaan dan mencoba mengubah angka statistik (seperti jumlah *node* yang berhasil dibuat), *Mutex* memastikan bahwa hanya ada satu proses yang boleh mencatat data pada satu waktu.

Potongan kode berikut diambil dari file `src/build.go`, yang menunjukkan bagaimana setiap anak *node* diproses secara bersamaan di dalam *goroutine* baru:

```go
func BuildOctreeConcurrent(node *OctreeNode, vertices []Vector3, faces []Face,
    currentDepth, maxDepth int, stats *OctreeStats, wg *sync.WaitGroup) {
    defer wg.Done()

    stats.mu.Lock()
    stats.CreatedNodesByDepth[currentDepth]++
    stats.mu.Unlock()
    ...
    var childWg sync.WaitGroup
    for i := 0; i < 8; i++ {
        ...
        if len(intersectingFaces) > 0 {
            childWg.Add(1)
            go BuildOctreeConcurrent(childNode, vertices, intersectingFaces,
                currentDepth+1, maxDepth, stats, &childWg)
        }
    }

    childWg.Wait()
}
```

7.2 Interactive Web Viewer

Untuk menampilkan model 3D, program ini tidak menggunakan aplikasi tambahan dari luar agar mudah dijalankan di komputer mana saja. Tampilan dibuat dalam bentuk halaman *web* interaktif menggunakan pustaka bawaan Go (`net/http`) yang dipadukan dengan fitur kanvas HTML5 dan JavaScript.

Agar tampilan 3D berjalan lancar dan tidak patah-patah saat diputar, program ini menerapkan dua strategi optimasi utama:

1. Program Go di belakang layar (diimplementasikan pada file `src/optimizer.go` dan `src/viewer.go`) akan mengecek setiap kubus *voxel*. Jika ada sisi kubus yang saling menempel dengan kubus tetangganya, sisi tersebut tidak akan dikirim ke *browser* karena posisinya tertutup di bagian dalam objek. Cara ini sangat menghemat ukuran data.

2. Perhitungan matematika untuk memutar titik-titik 3D dilakukan langsung oleh JavaScript di *browser* pengguna. Setelah itu, program menerapkan *Painter's Algorithm*. Algoritma ini bekerja dengan cara mengurutkan jarak kedalaman (sumbu Z) dari setiap sisi persegi. Sisi yang lokasinya paling jauh di belakang akan digambar dan diwarnai padat lebih dulu. Dengan begitu, sisi yang ada di depan akan otomatis menutupi garis-garis yang ada di belakangnya secara rapi.

Potongan kode JavaScript berikut diambil dari file `src/viewer.go`, yang mengatur cara gambar diproyeksikan dan diwarnai:

```javascript
const projected = model.Vertices.map(v => {
    const x1 = v.X * cosY - v.Z * sinY;
    const z1 = v.X * sinY + v.Z * cosY;
    const y1 = v.Y;
    const y2 = y1 * cosX - z1 * sinX;
    const z2 = z1 * cosX + y1 * sinX;
    return { x: x1 * scale + cx, y: -y2 * scale + cy, z: z2 };
});

facesToDraw.sort((a, b) => a.zDepth - b.zDepth);

facesToDraw.forEach(faceObj => {
    const face = faceObj.indices;
    ctx.beginPath();
    ctx.moveTo(projected[face[0]].x, projected[face[0]].y);
    for (let i = 1; i < face.length; i++) {
        ctx.lineTo(projected[face[i]].x, projected[face[i]].y);
    }
    ctx.closePath();

    ctx.fillStyle = '#ecf0f1';
    ctx.fill();
    ctx.stroke();
});
```

8 Pengujian

Untuk memastikan program berjalan dengan benar, dilakukan beberapa pengujian dengan beberapa kasus uji (*test case*. Kasus uji yang digunakan adalah sebagai berikut.

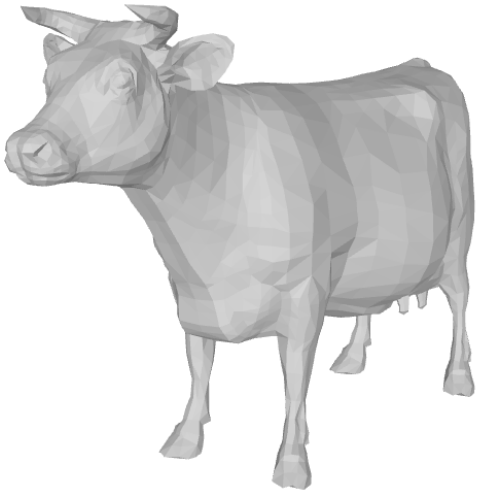
1. Kasus Uji `maxDepth` = 3

2. Kasus Uji `maxDepth` = 5

3. Kasus Uji `maxDepth` = 7

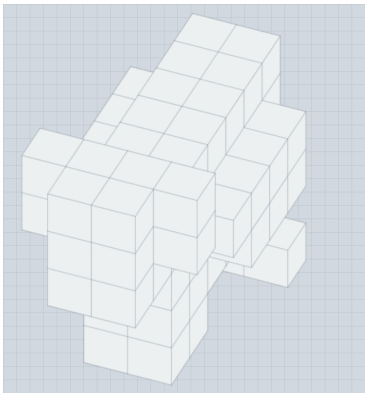
8.1 Kasus Uji Objek *cow*

- Input :



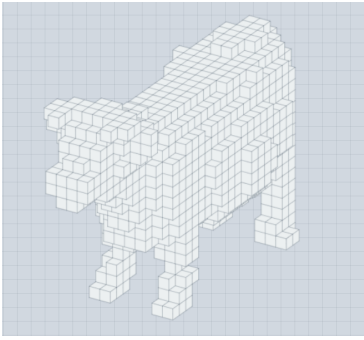
Gambar 3: Input Kasus Uji *cow*

- Output:
  1. `maxDepth` = 3



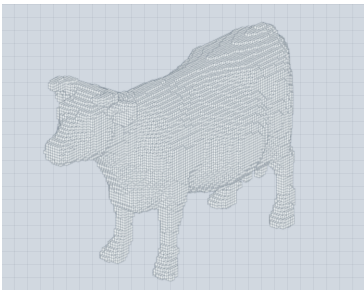
Gambar 4: Hasil Kasus Uji *cow* dengan `maxDepth` = 3

  2. `maxDepth` = 5



Gambar 5: Hasil Kasus Uji *cow* dengan `maxDepth` = 5

3. `maxDepth` = 7



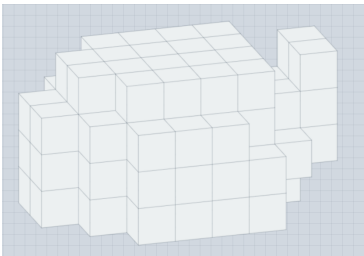
Gambar 6: Hasil Kasus Uji *cow* dengan `maxDepth` = 7

8.2 Kasus Uji Objek *teapot*

- Input :

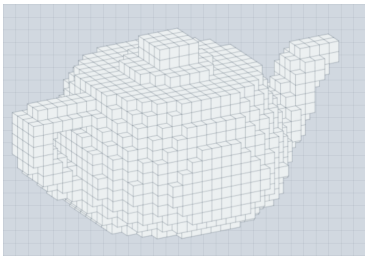
Gambar 7: Input Kasus Uji *teapot*

- Output:
  1. `maxDepth` = 3



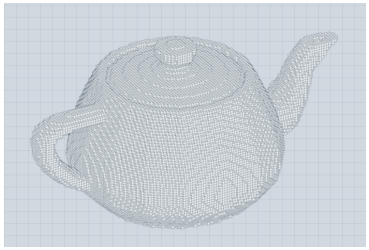
Gambar 8: Hasil Kasus Uji *teapot* dengan `maxDepth` = 3

  2. `maxDepth` = 5



Gambar 9: Hasil Kasus Uji *teapot* dengan `maxDepth` = 5

3. `maxDepth` = 7



Gambar 10: Hasil Kasus Uji *teapot* dengan `maxDepth` = 7

8.3 Hasil Pengujian

Berikut adalah ringkasan hasil pengujian kasus uji yang sudah dilakukan beserta jumlah *voxel*, *vertex*, dan *faces* yang terbentuk.

No	Deskripsi Kasus	Voxel	Vertex	Faces
1	*cow* `maxDepth` = 3	88	704	528
2	*cow* `maxDepth` = 5	1362	10896	8172
3	*cow* `maxDepth` = 7	22864	182912	137184
4	*teapot* `maxDepth` = 3	108	864	648
5	*teapot* `maxDepth` = 5	1748	13984	10488
6	*teapot* `maxDepth` = 7	29565	236520	177390

Tabel 1: Ringkasan Hasil Pengujian Kasus Uji

9 Penutup

9.1 Lampiran

Tautan kode sumber yang saya rancang dalam program ini ada pada tautan berikut.
Link Github: https://github.com/bangedaw/Tucil2_13524036_13524044.git

Rangkuman hasil program ini tertera pada tabel berikut.

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan *concurrency*	✓	
7	[Bonus] Program dapat menampilkan visualisasi .obj	✓	

Tabel 2: Ringkasan Keterselesaian Program

9.2 Kata Penutup

Jika saya berkenan untuk berkata jujur, sebaiknya pekerjaan akademik tidak diturunkan saat liburan/cuti bersama. Mari kita lebih menghargai cuti bersama, agar kelak saat bekerja nanti tidak diperbudak oleh perusahaan yang menyuruh bekerja bahkan saat liburan. Mungkin bisa berkaca dengan saudara mata kuliah ini, yaitu IF2224 Teori Bahasa Formal dan Otomata yang mengakui keberadaan liburan dan memberikan waktu kepada peserta mata kuliah untuk berlibur sebelum mengerjakan tugas. Terlepas dari itu, tugas ini seru banget! Terima kasih IF2211 Strategi Algoritma!

9.3 Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdesan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Bandung, 28 Maret 2026

Edward David Rumahorbo
13524036

Narendra Dharma Wistara M.
13524044